

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory works No. 1 & 2

Operating Systems

Lab 1: The OS as a Resource Manager

Lab 2: Meet the OS You Will Build

FAF-241, Ciur Alexandrina

Checked by:
Reitman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

1 Lab 1: The OS as a Resource Manager

1.1 Scopul Lucrării

Observarea directă din terminal a modului în care nucleul unui sistem de operare Linux acționează ca un manager de resurse interpus între procesele utilizator și componentele hardware. Laboratorul acoperă cele patru responsabilități majore ale sistemului de operare: gestiunea fișierelor, planificarea proceselor, administrarea memoriei și abstractizarea dispozitivelor de intrare/ieșire.

1.2 Informații Inițiale despre Sistem

```
$ whoami
user

$ uname -a
Linux user-VPCEH1L0E 6.8.0-138-generic #138~22.04.1-Ubuntu SMP
    PREEMPT_DYNAMIC Fri Aug  7 13:43:15 UTC  x86_64 x86_64 x86_64 GNU/
    Linux

$ uptime
 15:58:23 up 11 days, 21:52,  1 user,  load average: 1.34, 1.38, 1.37
```

1.3 Part 1: Gestiunea Fișierelor și Directoarelor

1.3.1 Comenzi rulate și output relevant

```
$ cd ~/os-lab1
$ mkdir demo && cd demo
$ echo "hello operating systems" > note.txt
$ cp note.txt copy.txt
$ mv copy.txt renamed.txt
$ ls -l
total 8
-rw-rw-r-- 1 user user 24 Sep 28 15:59 note.txt
-rw-rw-r-- 1 user user 24 Sep 28 15:59 renamed.txt
$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 user user 24 Sep 28 15:59 note.txt
```

1.3.2 Observații și Răspunsuri (Part 1)

1. **Proprietar și grup:** Fișierele create (`note.txt`, `renamed.txt`) aparțin utilizatorului `user` și grupului `user`.

2. **Semnificația celor 10 caractere după `chmod 600` (`-rw----`):**

   - Primul caracter (`-`) identifică entitatea drept fișier obișnuit (*regular file*).
   - Următoarele trei caractere (`rw-`) indică permisiunile utilizatorului proprietar: citire (`r`) și scriere (`w`), fără drept de execuție (`-`).
   - Următoarele trei caractere (`--`) arată lipsa oricăror permisiuni pentru membrii grupului.

- Ultimele trei caractere (`--`) arată lipsa oricăror permisiuni pentru restul utilizatorilor (*others*).

3. **Două directoare direct sub `/`:**
   - `/etc`: Găzduiește fișierele și directoarele de configurare globale ale sistemului.
   - `/dev`: Conține nodurile speciale de fișiere (*device files*) prin care kernelul expune hardware-ul fizic și virtual către spațiul utilizator.

1.4 Part 2: Gestiunea Proceselor

1.4.1 Comenzi rulate și output relevant

```
$ ps aux | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME
   COMMAND
root           1  0.1  0.3 168476 13912 ?        Ss   Sep16  33:34 /sbin
   /init splash
root           2  0.0  0.0      0     0 ?        S    Sep16   0:01 [
   kthreadd]
root           3  0.0  0.0      0     0 ?        S    Sep16   0:00 [
   pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Sep16   0:00 [
   kworker/R-rcu_g]

$ ps aux | wc -l
176

$ cat /proc/$PID/status | head -3
Name:   sleep
Umask:  0002
State:  S (sleeping)
```

1.4.2 Observații și Răspunsuri (Part 2)

1. **Identificarea procesului cu PID 1:** Procesul este `/sbin/init` (care este un link către `systemd`), fiind procesul de inițializare lansat de kernel la pornire și părintele direct sau indirect al tuturor proceselor din spațiul utilizator.

2. **Număr de procese active pe VM:** Comanda `ps aux | wc -l` a raportat 176 de linii (175 de procese plus linia de antet), iar utilitarul `top` a indicat un număr de 174 de procese gestionate de nucleu.

3. **Starea procesului din `/proc/<PID>/status`:** Linia indică `State:  S (sleeping)`, reprezentând starea de *interruptible sleep* în care procesul așteaptă pasiv expirarea unui cronometru sau recepționarea unui semnal, fără a consuma timp CPU.

1.5 Part 3: Gestiunea Memoriei

1.5.1 Comenzi rulate și output relevant

```
$ free -h
               total        used        free      shared  buff/cache
   available
Mem:           3.8Gi       615Mi       195Mi       4.0Mi       3.0Gi
       2.9Gi
Swap:          2.0Gi       204Mi       1.8Gi

$ grep VmRSS /proc/$PID/status
VmRSS:      2036 kB
```

1.5.2 Observații și Răspunsuri (Part 3)

1. **Memorie RAM totală și liberă:** Mașina virtuală dispune de o memorie RAM totală de **3.8 GiB**, din care **195 MiB** sunt complet nealocați (*free*), iar **2.9 GiB** sunt raportați ca disponibili efectiv (*available*) datorită memoriei cache eliberabile.

2. **Definiția memoriei Swap și cantitatea configurată:** Swap-ul reprezintă o zonă pe disc (în cazul curent fișierul `/swapfile`) folosită ca extensie a memoriei fizice pentru paginile inactive; sistemul are configurat un spațiu de **2.0 GiB** de swap, dintre care 204 MiB sunt folosiți.

3. **Memoria rezidentă (VmRSS) a procesului `sleep`:** Valoarea este de **2036 kB** ($\sim$2 MiB). Această dimensiune minimală este de așteptat, întrucât binarul `sleep` încarcă doar biblioteca standard C (`libc`) și apelează direct un apel de sistem de pauză fără a aloca structuri suplimentare de date.

1.6 Part 4: Gestiunea Dispozitivelor și Stocării

1.6.1 Comenzi rulate și output relevant

```
$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda      8:0    0 298.1G  0 disk
|-sda1   8:1    0   512M  0 part /boot/efi
|-sda2   8:2    0     1K  0 part
`-sda5   8:5    0 297.6G  0 part /var/snap/firefox/common/host-hunspell
                                 /
sr0     11:0    1  1024M  0 rom

$ ls -l /dev | head -4
crw-r--r-- 1 root root     10, 235 Sep 22 13:52 autofs
drwxr-xr-x 2 root root         640 Sep 28 03:08 block
drwxr-xr-x 2 root root          80 Sep 16 18:06 bsg
crw------- 1 root root     10, 234 Sep 16 18:06 btrfs-control
```

1.6.2 Observații și Răspunsuri (Part 4)

1. **Dispozitivul pe care este montat `/`:** Sistemul de fișiere rădăcină (`/`) este montat pe partiția de disc `/dev/sda5`.

2. **Exemplu de intrare din `/dev`:** Fișierul special de tip bloc `sr0` (asociat legăturii simbolice `cdrom`) reprezintă unitatea optică CD/DVD-ROM.

3. **Semnificația conceptului „everything is a file”:** Paradigma specifică sistemelor UNIX abstractizează dispozitivele hardware, canalele de comunicație și structurile nucleului sub formă de fișiere într-un arbore unificat, permițând manipularea lor uniformă prin apelurile de sistem clasice: `open`, `read`, `write` și `close`.

1.7 Concluzii

Sistemul de operare îndeplinește rolul de manager de resurse prin structurarea eficientă a **fișierelor** într-o ierarhie accesibilă, explorată cu comanda `ls`; prin planificarea și izolarea **proceselor** pe nucleele procesorului, monitorizate activ prin utilitarele `ps` și `top`; prin distribuirea dinamică a **memoriei** RAM și a spațiului de swap, evaluate prin comanda `free`; și prin abstractizarea unificată a **dispozitivelor** hardware și de stocare, identificate în sistem prin intermediul comenzii `lsblk`, conform filozofiei UNIX.

2 Lab 2: Meet the OS You Will Build

2.1 Scopul Lucrării

Familiarizarea cu sistemul de operare educațional **xv6** (o reimplementare a Unix Version 6 pe arhitectura RISC-V) rulat în emulatorul QEMU. Laboratorul implică compilarea nucleului, explorarea mediului de execuție, analiza codului sursă din directoarele `kernel/` și `user/`, precum și implementarea primului program utilitar în spațiul utilizator: `sleep.c`.

2.2 Part 1 & 2: Rulare și Explorare xv6

2.2.1 Comenzi rulate în interiorul nucleului xv6

În cadrul sesiunii interactive din consola xv6 au fost rulate utilitarele de bază integrate:

```
$ ls
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
```

2.2.2 Observații (Part 2)

1. **Trei programe incluse cu xv6:** `ls`, `cat`, `echo` (precum și `grep`, `wc`, `sh`).
2. **Două funcționalități esențiale ale SO pentru funcționarea conductelor (*pipes*):**
   - Suportul pentru procese multiple și execuție concurentă (crearea proceselor prin `fork` și planificarea lor).
   - Mecanismul de comunicație inter-procese (IPC) gestionat în nucleu prin apelul `pipe`, descriptori de fișiere (*file descriptors*) și redirectarea fluxurilor de date (`dup`, `close`).
3. **Comparație între shell-ul xv6 și shell-ul Linux:** Shell-ul din xv6 este o variantă minimală didactică ce asigură facilitățile fundamentale (execuție, redirectări I/O și pipes), lipsindu-i funcționalitățile moderne din Bash precum istoricul comenzilor, autocompletarea prin tasta Tab sau variabilele de mediu.

2.3 Part 3: Analiza Codului Sursă

2.3.1 Inspectarea apelurilor de sistem

Pentru analiza modului în care programele din spațiul utilizator invocă nucleul, s-au executat următoarele comenzi în directorul gazdă `~/xv6-riscv`:

```
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

Output-ul pentru localizarea funcțiilor a returnat:

```
69:sys_read(void)
83:sys_write(void)
```

2.3.2 Observații (Part 3)

1. **Apeluri de sistem utilizate în `user/cat.c`:**
   - `read()`: Solicită nucleului să citească un flux de octeți dintr-un descriptor de fișier într-un buffer din memoria procesului.
   - `write()`: Solicită nucleului să transfere octeții dintr-un buffer către un descriptor de fișier (ex. descriptorul 1 pentru *stdout*).
   - `exit()`: Notifică nucleul despre terminarea execuției procesului curent, eliberând structurile și resursele asociate.
2. **Implementarea funcției `sys_read`:** Funcția este implementată în fișierul `kernel/sysfile.c`, începând cu linia **69**.
3. **Diferența dintre codul din `kernel/` și `user/`:** Codul din `kernel/` se execută în modul privilegiat (*supervisor mode*) cu acces direct la registrele hardware și memoria fizică, în timp ce codul din `user/` rulează în mod neprivilegiat (*user mode*), interacțiunea cu resursele fiind mediată strict prin apeluri de sistem.

2.4 Part 4: Implementarea Utilitarului `sleep`

2.4.1 Codul sursă: `user/sleep.c`

A fost creat programul utilitar `sleep`, care primește numărul de tick-uri de ceas ca argument și apelează apelul de sistem existent:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  sleep(atoi(argv[1]));
  exit(0);
}
```

2.4.2 Configurarea build-ului în `Makefile`

În fișierul `Makefile`, binarul a fost adăugat în lista `UPROGS`:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    $U/_forktest\
```

2.4.3 Testarea programului

În consola xv6, testarea comportamentului s-a efectuat prin comenzile:

```
$ sleep 10
$ sleep
usage: sleep <ticks>
```

Comanda `sleep 10` a suspendat execuția pentru durata specificată, iar invocarea fără parametri a afișat corect mesajul de utilizare.

2.5 Concluzii

În cadrul lucrării a fost compilat și rulat nucleul xv6 în QEMU, au fost explorate utilitarele de bază și mecanismul de pipes, s-a analizat traseul unui apel de sistem de la programul utilizator (`user/cat.c`) până la implementarea sa în nucleu (`kernel/sysfile.c`) și s-a evidențiat separarea dintre modul utilizator și modul supervizor. Implementarea utilitarului `sleep` a demonstrat modul în care un program nou este integrat în sistem prin `Makefile` și cum acesta folosește un apel de sistem existent.